RoboSim AI

Technical Assignment — Mid-Level Full Stack Engineer

June 19, 2026

Project: RoboSim Scene Manager

CONTEXT

RoboSim AI builds tools for robotics simulation and digital twins. In this assignment, you will build a small full-stack web app for creating and managing simple 3D simulation scenes.

This is not expected to be a full simulator. We are evaluating your ability to build a clean, maintainable full-stack application with a Python backend, React-based frontend, 3D/WebGL viewer, and reproducible local deployment.

Expected duration: *1–2 days.*

GOAL

Build a web application where users can create, view, edit, and save simple robotics simulation scenes.

A scene contains:

- name
- description
- list of objects
- action/event history

Each object should have:

- id

- type: robot, box, shelf, conveyor, obstacle, etc.
- position: x, y, z
- rotation
- scale
- metadata/properties

Required Features

Frontend

Use any React-based stack.

Required:

- Scene list page
- Scene detail/editor page
- 3D scene viewer using Three.js, React Three Fiber, Babylon.js, or similar
- Add at least 3 object types
- Select an object in the scene
- Edit selected object properties
- Save scene changes
- Show recent scene events/logs

The UI should be clean and usable, but it does not need to be overly polished.

Backend

Use Python.

You may choose the framework and database.

Required API features:

- Create scene
- List scenes
- Get scene by ID
- Update scene
- Delete scene
- Add/update/delete scene objects
- Retrieve scene event logs

The API should include validation and clear error responses.

Local Deployment

The project must be easy to run locally.

Required:

- Backend Dockerfile
- Frontend Dockerfile
- Docker Compose file
- Clear setup instructions in README

Bonus:

- GitHub Actions or GitLab CI
- Local Terraform/OpenTofu setup
- Tests
- Scene import/export as JSON

Deliverables

Submit:

- Git repository
- README with:
  - setup instructions
  - architecture overview
  - API overview
  - design decisions
  - known limitations
  - improvements you would make with more time
- Short demo video, 3–7 minutes, showing:
  - local setup
  - main features
  - 3D viewer
  - backend/API behavior

Evaluation Criteria

We will evaluate:

- Code quality
- Backend API design

- Frontend architecture
- 3D/WebGL implementation
- UX clarity
- Data modeling
- Docker/local deployment quality
- Error handling
- README quality
- Engineering judgment
- Ability to explain your code

AI coding tools are allowed, but you must understand the code. Technical questions about implementation choices should be expected.

NOTES

- Do not implement authentication.
- Do not over-engineer the project.
- A smaller, polished, well-structured solution is better than a large unfinished one.